

TP2 : Les outils Gcc et make

1. Démarrage

L'objectif de ce TP est de se familiariser avec le langage C et sa chaîne de développement basée sur le compilateur gcc, et de vous apprendre à créer et utiliser des makefiles lorsque vous développez des programmes en C.

2. Un programme en C composé d'un seul fichier de compilation

Créez un nouveau répertoire pour ce laboratoire dans votre espace personnel et copiez le contenu du répertoire /share/l3info/CUnix/tp2 en utilisant la commande `cp -r`.

1- À l'intérieur du répertoire copié, compilez le programme `hms.c` en utilisant la commande :
`gcc -Wall nom_fichier.c`

Que signifient les options `-Wall` ?

2- Ouvrez le fichier source avec un éditeur et corrigez les erreurs.

3- Recompilez le programme pour vérifier que le programme est désormais correct.

4- Utilisez la commande `ls -ail` pour identifier le fichier produit en résultat du processus de compilation. Quel type de fichier est-ce ?

5- Pour spécifier un nom d'exécutable particulier pour le fichier produit, vous pouvez utiliser l'option `-o` de gcc :

`gcc -o nom_prog -Wall nom_fichier.c`

Nommez le fichier exécutable de `hms.c` comme `hms` (les fichiers exécutables sous Linux n'ont pas l'extension `.exe` comme dans DOS / Windows).

6- Exécutez le programme en utilisant `./` et vérifiez que sa fonctionnalité est correcte.

3. Un programme C composé de plusieurs fichiers de compilation

En règle générale, un fichier de compilation se compose d'un fichier avec l'extension `.c` contenant le code source et d'un fichier d'en-tête avec l'extension `.h`, qui permet d'exporter les définitions des identifiants du

module (données, fonctions, etc.).

Lorsque nous compilons un fichier de compilation, nous utilisons l'option `-c` du compilateur. Cela produit un fichier objet avec l'extension `.o` qui n'est pas directement exécutable.

1- Accédez au répertoire `"tp_liste"`, que vous avez déjà copié lors de l'exercice précédent. Ce répertoire contient le répertoire `"src"` où sont stockés les fichiers source et le répertoire `"include"` où sont stockés les fichiers d'en-tête. Les fichiers source contiennent le code source (partiel) de l'implémentation des listes chaînées en langage C.

2- Compilez les fichiers `src/test_list.c` à l'aide de la commande :

```
gcc -Wall -c src/test_list.c
```

3- Cela génère une erreur de compilation. D'où provient cette erreur ? Comment la corriger ?

- Index 1 : nous avons un fichier d'en-tête `list.h` qui définit les variables et les identifiants exportés par `list.c`, stocké dans le répertoire `"include"`, et la directive `#include "list.h"`.
- Index 2 : il est parfois nécessaire d'indiquer au compilateur où trouver les fichiers d'en-tête s'ils ne se trouvent pas dans le même répertoire. Pour ce faire, nous pouvons utiliser l'option `-I` de gcc :

```
gcc -I./include
```

Cela indique de chercher les fichiers d'en-tête `.h` dans le répertoire `./include`.

4- Une fois le problème résolu, compilez (avec l'option `-c`) le fichier **`src/list.c`**. Quelle est la sortie de cette commande ? Ce fichier est-il exécutable ?

5- Connectez les deux fichiers objets pour obtenir le programme exécutable final en utilisant la commande :

```
gcc -o test_list test_list.o list.o
```

6- Exécutez le programme et observez les résultats sur la sortie standard.

7- Le message d'erreur `"segmentation fault"` indique que le programme a tenté d'accéder à une adresse mémoire illégale, soit en raison d'un débordement de tableau, soit de l'utilisation d'un pointeur incorrect ou non initialisé.

8- Recompilez le fichier **`test_list.c`** en utilisant l'option `-Wall`. Quel résultat obtenez-vous ?

9- En vous basant sur les avertissements fournis par le compilateur, corrigez les erreurs dans le module **`test_list.c`**, puis vérifiez le bon fonctionnement du programme.

4. L'outil make

Chaque fois que vous modifiez l'un des fichiers de compilation (`.c` ou `.h`), vous devez recompiler tous les fichiers de compilation dépendants et refaire la liaison entre les fichiers objets pour obtenir un fichier exécutable à jour. Cette tâche est très chronophage si elle est effectuée manuellement, surtout lorsque le programme comprend de nombreux fichiers de compilation (par exemple, le noyau du système d'exploitation Linux qui contient plusieurs centaines de fichiers de compilation C).

Le programme `make` permet d'automatiser cette tâche en recompilant uniquement les fichiers nécessaires à chaque fois.

1- Complétez le fichier makefile du répertoire tp2 pour permettre la compilation correcte et la liaison des fichiers de compilation **test_list.c** et list.c selon les instructions suivantes :

Les fichiers .o produits lors de la compilation sont stockés dans le répertoire **./obj**.

Le programme exécutable s'appelle test_list et est stocké dans le répertoire **./bin**.

Les fichiers objets et l'exécutable sont à jour par rapport à leurs fichiers source .c et fichiers d'en-tête .h dépendants.

2- Vérifiez le bon fonctionnement de votre makefile en modifiant l'un des fichiers, puis relancez le makefile en utilisant la commande make.

3- Dans le fichier makefile, complétez l'entrée clean afin de supprimer tous les fichiers objets .o et également l'exécutable produit.

4- Dans le fichier makefile, complétez l'entrée listing afin de produire un fichier PDF à partir des sources des fichiers .c et .h, en utilisant la séquence de commandes suivante :

```
a2ps -o listing.ps include/list.h src/list.c src/test_list.c
ps2pdf listing.ps
rm listing.ps
```

5- Vérifiez que cette règle fonctionne correctement en l'appelant avec la commande :

```
make listing
```

6- Si vous souhaitez développer un programme en utilisant un "débugueur", vous devez le compiler (et tous ses modules) avec l'option -g, comme indiqué ci-dessous :

```
gcc -Wall -g -o test_list test_list.o list.o
```

Pour éviter de devoir modifier toutes les règles du makefile à chaque fois, vous pouvez définir des variables pour être plus générique. Par exemple, en ajoutant la ligne suivante au début du makefile, vous pouvez définir une variable CFLAGS.

```
CFLAGS = -Wall -g
```

Ensuite, vous pouvez faire référence à cette variable en écrivant \$(CFLAGS) dans les règles du makefile, comme dans l'exemple ci-dessous :

```
gcc $(CFLAGS) -c module1.c
```

Modifiez le makefile pour que les appels à gcc soient effectués avec l'option -g.